

DMSF: A Dynamic Model Splitting Framework for Edge-Cloud Collaborative Inference

Xinyun Zhang*, Li Wang*, Xin Wu*, Lianming Xu[†], Yingyan Hou[‡], Yi Jiang[‡] and Aiguo Fei*

*School of Computer Science, Beijing University of Posts and Telecommunications, China

[†]School of Electronic Engineering, Beijing University of Posts and Telecommunications, China

[‡]Aerospace Information Research Institute, Chinese Academy of Sciences, China.

Email: {zhangxinyun, liwang, xinwu, xulianming, aiguofoei}@bupt.edu.cn, houyy@aircas.ac.cn, jiangyi22@mails.ucas.ac.cn

Abstract—Edge-cloud collaborative inference is a widely adopted approach in edge intelligence, especially for latency-sensitive tasks such as drone inspection, augmented reality, and disaster relief. To adapt to varying network bandwidth and limited computational capacity, model splitting methods divide Deep Neural Networks (DNNs) at specific split points into edge and cloud segments. However, current methods suffer from high switch latency and memory overhead, and are incompatible with Feature Pyramid Network (FPN)-based multi-branch models. To address the above issues, we propose a dynamic model splitting framework (DMSF) for edge-cloud collaborative inference. We design switchable compress–recover modules after each layer, enabling a single model to support all split points and significantly reduce memory overhead. The optimal split point is selected based on bandwidth and edge computational capacity, and the corresponding compress–recover path is activated for fast switching. For FPN-based multi-branch models, we remove the multi-scale branches from the backbone to the FPN and present hierarchical compensation modules in DMSF, making the model more suitable for splitting while maintaining perception accuracy. Experimental results show that DMSF reduces memory overhead by 75%, achieves millisecond-level switching, and outperforms the existing method with 32.0% lower latency and 12.2% higher perception accuracy.

Index Terms—Edge-cloud collaborative inference, model splitting, FPN

I. INTRODUCTION

IN recent years, with the rapid advancement of edge computing and artificial intelligence technologies, edge intelligence has been widely adopted in real-time perception and decision-making tasks, such as drone detection [1], augmented reality [2], and disaster relief [3]. Edge-cloud collaborative inference is an effective solution in edge intelligence, where deep neural networks (DNNs) are partitioned into two stages: the edge performs initial feature extraction, while the cloud completes the remaining computation. This collaboration helps reduce latency and alleviate the computational burden on edge devices. Due to network bandwidth variation and limited computational capacity, edge-cloud collaborative inference still faces challenges such as high transmission latency and difficulty adapting to dynamic environments.

This work was supported in part by the Key Laboratory of Target Cognition and Application Technology 2023-CXPT-LC-005, in part by the National Natural Science Foundation of China under Grants 62525106, 62171054, 62201089, and 62201071, and in part by the Natural Science Foundation of Beijing Municipality under Grant L222041, in part by the Fundamental Research Funds for the Central Universities under Grants 2024RC06, in part by China Unicom Research Institute 2025-STHZ-BJYDDX-004. (Corresponding author: Xin Wu (xinwu@bupt.edu.cn)).

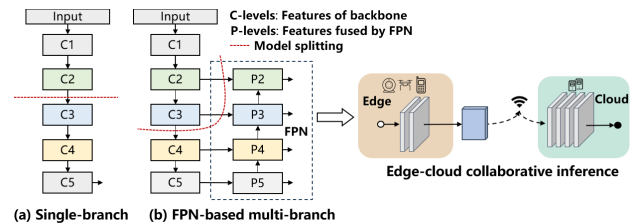


Fig. 1. Single-branch and FPN-based multi-branch model structures.

Existing edge-cloud collaborative inference methods reduce the volume of transmitted features and adapt to dynamic network conditions through feature compression and split point selection. Feature compression methods typically adopt a fixed split point and compress intermediate features using techniques such as JPEG, Huffman coding, or autoencoders [4]. However, static splitting lacks adaptability to dynamic network conditions. For example, in emergency response scenarios with fluctuating uplink bandwidth, fixed split points can lead to suboptimal bandwidth utilization and degraded performance. Although some approaches [5] improve flexibility by training separate compression models for each candidate split point, this design increases memory usage and switching latency on edge devices, limiting overall efficiency and adaptability.

Some studies have explored dynamic split point selection based on real-time bandwidth and computational capacity to balance transmission latency and edge computation. While early methods [6], [7] focused on simple single-branch models with clear layer-wise structures, others [8] extended to multi-branch models like InceptionNet [9] by applying graph-cut techniques, leveraging the relative independence of their branches. Notably, many modern visual models, such as those based on Feature Pyramid Networks (FPN), adopt complex multi-branch architectures with dense horizontal and cross-layer connections for enhanced multi-scale feature representation. As illustrated in Fig. 1, split points often span multiple feature paths, leading to feature redundancy and increased transmission overhead. Consequently, two key challenges remain for existing methods: 1) how to efficiently and dynamically switch optimal split points under bandwidth fluctuation and limited computational capacities, and 2) how to maintain perception accuracy and stability during dynamic splitting.

To tackle these challenges, we propose a Dynamic Model Splitting Framework (DMSF) for edge-cloud collaborative inference, achieving low latency and memory overhead. Our

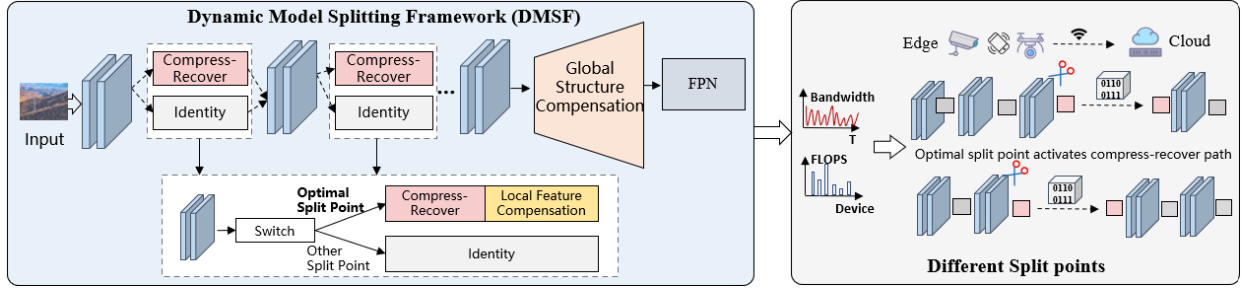


Fig. 2. The architecture of the proposed DMSF.

key contributions are summarized as follows:

- We propose DMSF for edge-cloud collaborative inference. We design switchable compress-recover modules after each layer, enabling a single model to support all candidate split points. The optimal split point is selected based on current bandwidth and edge computational capacity, and the corresponding compress-recover path is dynamically activated.
- We present hierarchical feature compensation modules in DMSF to preserve accuracy. For FPN-based models, global structure compensation restores low-level features after removing multi-scale branches, while local feature compensation aligns feature distributions at split points using mean and variance to enhance consistency.
- We evaluate DMSF on YOLOv5 object detection using the VisDrone dataset. DMSF reduces memory overhead by 75% and achieves millisecond-level switching latency compared to the independent split model. Under a 10 Mbps bandwidth on the TX2 platform, compared to the existing method SP-Reconfig [10], DMSF lowers latency by 32.0% and improves perception accuracy by 12.2%.

The rest of this paper is organized as follows. Section II details the problem formulation. The architecture of the proposed DMSF is in section III. Experiential results are conducted in section IV. Finally, section V draws a conclusion and prospect.

II. PROBLEM FORMULATION

In DNNs, deeper layers produce smaller spatial feature maps but with more channels, resulting in significantly less data volume than the original input. Due to variations in computational capacity and network bandwidth, selecting the optimal split point is essential to minimize the total latency of edge computation, feature transmission, and cloud computation. The problem is defined as follows:

$$\min_p \sum_{i=0}^p T_{edge}(i) + T_{trans}(p) + \sum_{i=p+1}^N T_{cloud}(i) \quad (1)$$

$$\text{where } T_{trans}(p) = \frac{S_p}{B},$$

$$\text{s.t. } 0 \leq p \leq N-1, \quad p \in \mathbb{Z},$$

where p is the optimal split point. S_p is the volume of feature maps by the p -th layer. B is the current bandwidth.

III. ARCHITECTURE OF THE PROPOSED DMSF

In this section, we first present an overview of our proposed DMSF. Next we describe the structural components of our designed switchable compress-recover modules. Then, we introduce the designed hierarchical feature compensation modules. Finally, we present the training and inference strategy of our proposed DMSF.

A. Overview of the proposed DMSF

Fig. 2 illustrates the overall architecture of DMSF. Each candidate split point is embedded with a switchable compress-recover module. The compress-recover module compresses intermediate features to reduce transmission overhead, while the identity module forwards features unchanged to the next layer. For each candidate split point, the corresponding compress-recover module is activated only when selected as optimal based on real-time bandwidth and edge computational capacity. For FPN-based multi-branch models, we remove the multi-scale branches from the backbone and retain only the main output path. We design hierarchical feature compensation modules that contain global structure compensation and local feature compensation. Global structure compensation module is placed at the end of the backbone, reconstructing low-level multi-scale features to FPN. Local feature compensation refines feature distributions at split points based on the mean and variance to restore feature compression loss.

B. Switchable compress-recover module

To enable dynamic splitting, we insert a switchable compress-recover module after each candidate split point in the backbone. This module consists of two parallel paths: a compress-recover path and an identity path. The compress-recover path reduces intermediate output feature size, while the identity path directly passes the features to the next layer. When a specific layer is selected as the optimal split point, its compress-recover path is activated. Otherwise, the identity path is used. Path selection is controlled dynamically during the forward pass. The compress-recover module performs compression and recovery on the intermediate output features at the selected optimal split point p .

Feature compression is first performed on the edge side through a set of 2×2 convolutional layers, which reduce the

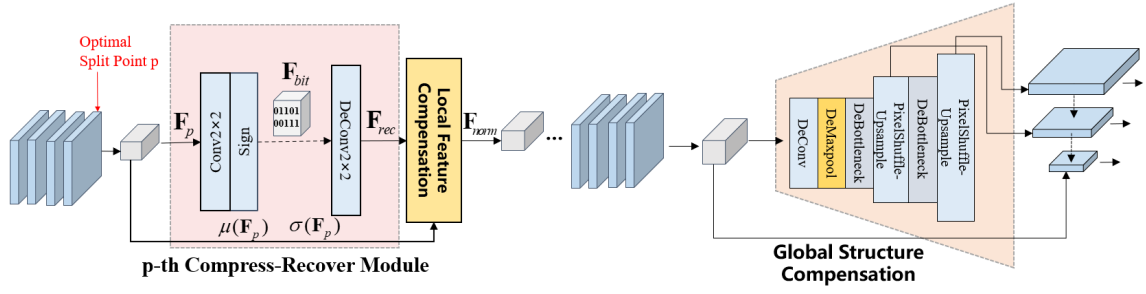


Fig. 3. Hierarchical feature compensation modules.

number of channels in the feature map while preserving essential spatial information. This channel-wise compression helps lower the data volume for transmission without significantly compromising the representational capacity of the features. Let the original feature tensor be $\mathbf{F}_p \in \mathbb{R}^{H \times W \times C}$, where H , W , C denotes the height of feature map, the width of feature map and the number of channels.

$$\mathbf{F}_{comp} = \sigma(\text{Conv}_{2 \times 2}(\mathbf{F}_p)), \quad \mathbf{F}_{comp} \in \mathbb{R}^{H \times W \times C'} \quad (2)$$

where $\mathbf{F}_{comp} \in \mathbb{R}^{H \times W \times C'}$ denotes the channel compressed feature. $\sigma(\cdot)$ denotes activation function. To further reduce transmission overhead, we apply 1-bit quantization to the compressed features. Using the Sign function, float32 values in the feature map are converted into 1-bit data, significantly lowering the transmission cost.

$$\mathbf{F}_{bit} = \text{Sign}(\mathbf{F}_{comp}), \quad \mathbf{F}_{bit} \in \{-1, +1\}^{H \times W \times C'} \quad (3)$$

where $\mathbf{F}_{bit} \in \mathbb{R}^{H \times W \times C'}$ denotes the quantified feature. $\text{Sign}(\cdot)$ denotes the Sign function, converting each floating-point value to -1 or $+1$ to achieve 1-bit representation.

Feature recovery module is performed on the cloud side using 2×2 transposed convolution. This operation not only restores the channel dimension, but also recovers the numerical precision of the feature map. Since the convolutional weights are stored in 32-bit floating point format, the 1-bit intermediate features can be directly multiplied with these weights, enabling accurate reconstruction of float32-level features without requiring additional decoding steps.

$$\mathbf{F}_{rec} = \text{DeConv}_{2 \times 2}(\mathbf{F}_{bit}), \quad \mathbf{F}_{rec} \in \mathbb{R}^{H \times W \times C} \quad (4)$$

where $\mathbf{F}_{rec} \in \mathbb{R}^{H \times W \times C}$ denotes the recovered features.

C. Hierarchical feature compensation modules

Fig. 3 shows the details of the hierarchical feature compensation modules, including global structure compensation and local feature compensation. The global structure compensation module is placed between the backbone and the FPN, passing the recovered low-level features to the FPN to maintain accuracy. The local compensation module is placed after the feature compress-recover module to alleviate feature loss during compression and quantization, improving the consistency of feature representation.

In the original model, the backbone extracts semantic information through layer-by-layer downsampling using convolutions and passes low-level features to the FPN for multi-scale processing. In the DMSF, the input to the global structure compensation module is the features extracted from the final layer of the backbone. The module employs a cascaded upsampling architecture to progressively compensate low-level features from high-level feature. Each upsampling operation is designed as the inverse of the corresponding downsampling operation in the backbone, enabling the recovery of original low-level feature details. If the operation at layer l in the backbone is denoted as $\mathcal{B}_l(\cdot)$, the corresponding upsampling module is defined as:

$$U_l(\cdot) = \text{UpSampleBlock}(\mathcal{B}_l^{-1}(\cdot)), l \in \mathcal{L} \quad (5)$$

where $\mathcal{B}_l^{-1}(\cdot)$ represents the inverse mapping operation. For the existing feature extraction components, we have implemented the corresponding upsampling modules. The main implementation strategy is as follows:

- **PixelShuffleUpsample:** The inverse mapping of convolution. This operation realizes $2 \times$ upsampling through channel reorganization, and reduces the computational complexity by 71% compared with deconvolution.
- **DeBottleneck:** The inverse mapping of the Bottleneck. We replace the standard convolution operations with transposed convolutions while preserving the residual connections. To mirror the backbone structure, the up-sampling path uses the same number of stacked DeBottleneck modules.
- **DeMaxPool:** The inverse mapping of the MaxPool. Since pooling operations in the feature extraction network do not have a precise inverse operation, we employ transposed convolution as an alternative.

Due to the global feature compensation module, the feature extraction network no longer needs to transmit multi-branch features to the cloud, but only needs to transmit single-layer features of the split point layer, reducing transmission overhead.

To enhance feature recovery capability, we design the local feature compensation based on the adaptive instance normalization (AdaIN) [11]. Originally proposed for style transfer, AdaIN aligns the mean and variance of content features with those of a target style, enabling the transfer

Algorithm 1 Adaptive Split Point Selection and Inference in DMSF

Input: Candidate split points set $P = \{1, 2, \dots, N\}$, current bandwidth B , edge-side latency $\{T_{edge}^i\}_{i \in P}$, cloud-side latency $\{T_{cloud}^i\}_{i \in P}$, output feature size $\{S_i\}_{i \in P}$, input image I , global compensation layer set $\mathcal{L} \in \mathcal{L}$.

Output: Optimal split point p^* ; Final prediction $\hat{y}$.

- 1: **for all** $i \in P$ **do**
- 2: Compute transmission time: $T_{trans}^i = \frac{S_i}{B}$;
- 3: Compute total latency: $T_{total}^i = T_{edge}^i + T_{trans}^i + T_{cloud}^i$;
- 4: **end for**
- 5: Select optimal split point: $p^* = \arg \min_{i \in P} T_{total}^i$;
- 6: **Edge Inference (up to p^*):**
Extract feature: $\mathbf{F}_p = f_{edge}^{[1:p^*]}(I)$;
- 7: Compute mean $\mu(\mathbf{F}_p)$ and variance $\sigma(\mathbf{F}_p)$ of original features $\mathbf{F}_p$;
- 8: Activate the corresponding p -th compress-recover module on the edge and cloud;
- 9: Transmit $\mathbf{F}_{bit}, \mu(\mathbf{F}_p)$ and $\sigma(\mathbf{F}_p)$ to the cloud;
- 10: Local compensation: $\mathbf{F}_{norm} = \mathcal{N}(\mathbf{F}_{rec}, \mu(\mathbf{F}_p), \sigma(\mathbf{F}_p))$;
- 11: **Cloud Inference (from p^*):**
Continue extract feature: $y = f_{cloud}^{[p^*:L]}(\mathbf{F}_{norm})$;
- 12: Global compensation: $y' = U_l(y) \mid l \in \mathcal{L}$;
- 13: Continue inference: $\hat{y} = f_{cloud}^{[L:]}(y')$;

of stylistic patterns. In our previous description, intermediate features are compressed and quantized through a high degree of channel, and even through feature recovery, it is difficult to be consistent with the distribution of original features. Low order statistics such as mean and variance contain semantic information about the intensity and distribution of feature activation. By adaptively adjusting the restored features to match the mean and variance of each channel of the original features, the original distribution features can be partially restored even if the high-frequency details are not explicitly modeled. This lightweight statistical alignment helps to reduce information inconsistency caused by lossy compression.

$$\begin{aligned} \mathbf{F}_{norm} &= \mathcal{N}(\mathbf{F}_{rec}, \mu(\mathbf{F}_p), \sigma(\mathbf{F}_p)) \\ &= \sigma(\mathbf{F}_p) \times \frac{\mathbf{F}_{rec} - \mu(\mathbf{F}_{rec})}{\sigma(\mathbf{F}_{rec})} + \mu(\mathbf{F}_p), \end{aligned} \quad (6)$$

where $\mathbf{F}_{norm} \in \mathbb{R}^{H \times W \times C}$ is the post-compensation features. $\mu(\mathbf{F}_p)$, $\sigma(\mathbf{F}_p)$ denotes the mean and variance of original features. $\mu(\mathbf{F}_{rec})$, $\sigma(\mathbf{F}_{rec})$ denotes the mean and variance of recovered features. Local feature compensation makes the recovered features more closely resemble the original features, helping to mitigate the effects of quantization and improve the accuracy of the recovered features.

D. Training and Inference of DMSF

Training: Before each training step, a split point is randomly sampled from the predefined set. During forward propagation, the corresponding compress-recover module is activated at the selected point, while identity modules are used for the others. The entire network is then optimized using the

TABLE I
THE EDGE DEVICES AND CLOUD DEVICE CONFIGURATION

Device	GPU	CPU	Memory
Jetson TX2	256 cores Pascal 1.33 TFLOPS	6-core@2.0GHz 2-Denver2+4-A57	8 GB
Jetson Nano	128 cores Maxwell 472 GFLOPS	4-core A57@1.43GHz Cortex-A57	4 GB
RTX 4090	16384 cores 82.58 TFLOPS	36-core@2.10GHz Xeon Gold 6338	24 GB

TABLE II
COMPARISON OF SWITCHING OVERHEAD BETWEEN INDEPENDENT SPLIT MODEL AND DMSF

Method	Storage (MB)	Memory (MB)	Latency(s)	
			TX2	Nano
Independent split model	21.2	4888	3.52	5.71
DMSF	11.6	1222	0.002	0.005

accumulated gradient from the sampled configuration. This unified sampling strategy ensures all split paths are sufficiently trained, improving the generalization of the DMSF.

Split point selection and online inference: After training, DMSF is deployed for collaborative inference with only a single model on the edge, avoiding the overhead of loading multiple sub-models. Considering resource constraints, only the backbone is deployed on the edge, while the full model remains on the cloud. The optimal split point is computed based on Equation 1. In the offline phase, we record the computation latency and output feature size of each candidate split point. During inference, the edge monitors bandwidth and selects the split point with the lowest estimated end-to-end delay. The corresponding compress-recover module is activated during forward process, as detailed in Algorithm 1.

IV. EXPERIMENTS AND RESULTS

In this work, we use the NVIDIA Jetson TX2 and Jetson Nano as edge devices, and an NVIDIA RTX 4090 GPU as the cloud device. The hardware specifications are summarized in Table I. To evaluate the proposed DMSF, we adopt the YOLOv5s [12] object detection based on an FPN architecture. The model is trained and tested on the VisDrone2019 [13] dataset, which focuses on aerial views captured by drones. During training, all input images are resized to 640×640 , and performance is evaluated on the validation set. The model is trained for 600 epochs with a batch size of 32. The evaluation includes the total end-to-end inference latency and accuracy expressed in mAP.

Table II compares the switching overhead of the independent split model and DMSF in terms of storage, memory usage, and switching latency on TX2 and Jetson Nano. DMSF requires only a single unified model of 11.6MB, while the independent model stores separate weights for each split point, totaling 21.2MB, resulting in a 45% reduction in storage. In terms of memory usage, DMSF loads only one model during runtime with a footprint of 1222MB, whereas the independent model runs multiple processes and loads several models simultaneously, consuming 4888MB of memory. This

TABLE III
ABLATION ANALYSIS OF THE PROPOSED DMSF

Method	Feature (KB)				Accuracy (%)
	3	5	7	10	
Multi-Branch	3200	4800	5600	5200	35.4
G-StructComp	3200	1600	800	400	37.4
Channel compression	800	400	200	100	31.4±0.8
1-bit quantization	25	12	6	3	31.2±0.8
L-FeatComp	26	15	11	6	32.4±0.8

corresponds to a 75% reduction in memory usage when using DMSF. On memory-limited devices like Jetson Nano, the independent model may even exceed the available memory. Regarding switching latency, the independent model suffers from delays due to the frequent loading and unloading of models. DMSF allows fast switching by updating parameters within a single model. It achieves a latency of 0.002 seconds on TX2 (compared to 3.52 seconds) and 0.005 seconds on Nano (compared to 5.71 seconds), reaching millisecond-level switching latency.

Table III presents the results of our ablation experiments on each split point of the proposed DMSF. The results include the output feature volume of each split point and the accuracy range of all split points. We choose the 3rd, 5th, 7th, and 10th layers of the model as candidate split points. The original YOLOv5s model, used as the baseline without any modifications, transmits several thousand kilobytes of feature data at each split point, achieving an accuracy of 35.4%. Notably, layers 5, 7, and 10 correspond to multi-branch feature layers. By applying the global structure compensation (G-StructComp) module, the redundant multi-branch transmission is significantly reduced. Specifically, at the 10th layer, the transmitted feature size drops to 400KB. The accuracy improves to 37.4%. This gain is particularly significant in small-object datasets like VisDrone, where global compensation recovers an additional feature layer, leading to a 2% improvement over the original model. To further reduce transmission overhead, we apply $4\times$ channel reduction, compressing intermediate features from 400KB to 100KB. Additionally, 1-bit quantization reduces the data precision from 32 bits to 1 bit, shrinking the size to just 3KB. However, such aggressive compression causes a notable drop in accuracy, with accuracy falling to 31.2%. To mitigate this, we introduce local feature compensation (L-FeatComp), which adds only lightweight statistics (mean and variance) to the transmitted data. This leads to a 1.2% improvement in accuracy with a negligible increase in feature size. These results confirm that DMSF can achieve a high compression ratio while effectively preserving perception performance.

Table IV compares the performance of edge-only inference and the proposed DMSF under a 5 Mbps bandwidth on devices with varying computational capabilities. In edge-only inference, the model runs entirely on the edge device. The accuracy remains at 35.4%, while latency increases as computational capability decreases, from 70.2ms on TX2 GPU to 1427.0ms on Nano CPU, indicating that limited resources

TABLE IV
EDGE-ONLY VS. DMSF UNDER VARYING COMPUTATIONAL CAPACITIES (5 MBPS BANDWIDTH)

Method	Device	Split point	Latency (ms)	Accuracy (%)
Edge	TX2 GPU	-	70.2	35.4
	Nano GPU	-	124.7	35.4
	TX2 CPU	-	827.5	35.4
	Nano CPU	-	1427.0	35.4
DMSF	TX2 GPU	10	72.4	31.7
	Nano GPU	5	99.9	33.2
	TX2 CPU	3	627.4	32.3
	Nano CPU	3	819.5	32.3

severely impact inference speed. With decreasing device performance, DMSF shifts the split point earlier in the model to minimize edge-side computation and maintain low latency. On TX2 GPU, layer 10 is selected to reduce transmission overhead, resulting in 72.4ms latency. On Nano GPU, layer 5 is used to balance edge computation and communication, reducing latency to 99.9ms. For low-power devices such as TX2 CPU and Nano CPU, layer 3 is selected to minimize edge-side load. On the highly resource-limited Nano CPU, DMSF reduces latency by 42.6% compared to edge-only inference, while maintaining acceptable accuracy with only a 3.1% drop. Although DMSF incurs a slight drop in accuracy, it significantly improves real-time performance. By balancing computation and communication, DMSF proves effective in resource-constrained edge environments.

Fig. 4 illustrates the inference latency and perception accuracy of different methods under varying bandwidth conditions (5 Mbps, 10 Mbps, 15 Mbps, and 30 Mbps) on the TX2 and Jetson Nano platforms. Fig. 4(a) presents the results on the TX2 platform, while Fig. 4(b) shows the results on the Nano platform. Four baseline methods are compared in both figures: 1)End-Only: the entire model is executed on the edge device; 2)Cloud-Only: the entire model is executed in the cloud and 640×640 images are transmitted; 3)SP-reconfigurable model (SP-Reconfig) [10]: implemented on the YOLOv5 object detection model by applying 1-bit quantization and convolution layer restoration at the selected split point. Considering the difficulty of detecting small objects, we adopt the accuracy-optimized scale model and reproduce the results on the VisDrone dataset; 4)DeepAdaIn-Net [4]: a static split method based on YOLOv5, also reproduced on the VisDrone dataset.

The results show that while End-Only and Cloud-Only methods maintain relatively high accuracy 35.4%, they suffer from significant limitations. End-Only places the entire computational burden on the edge, resulting in high latency, especially on low-performance devices like Nano. Cloud-Only requires transmitting full images to the cloud, leading to severe latency under low bandwidth (e.g., 265.8ms at 5Mbps), with unstable performance across bandwidths. SP-Reconfig supports adaptive split selection and applies 1-bit quantization with convolutional restoration. However, its use of batch normalization at each split point increases compu-

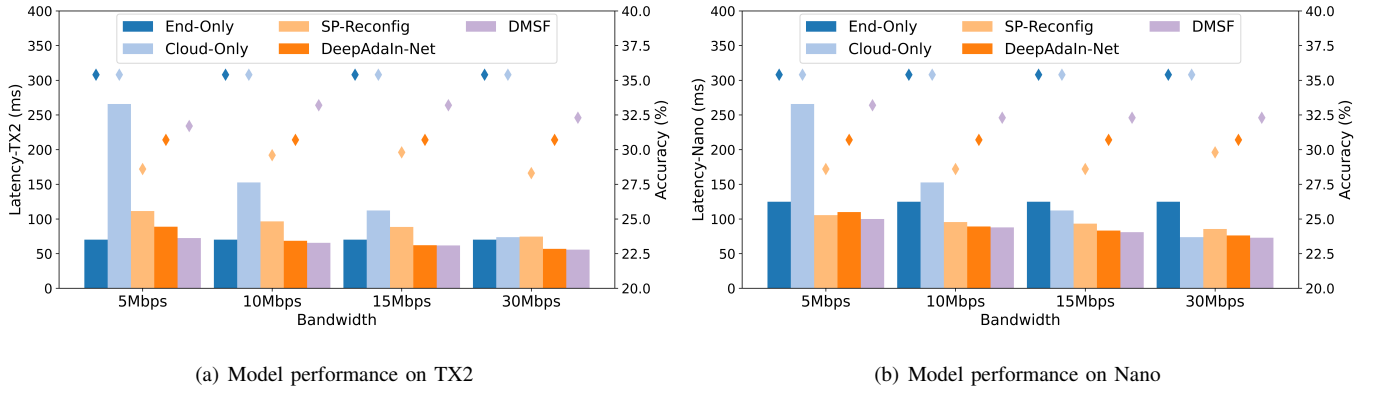


Fig. 4. Comparison under varying bandwidth. Latency is shown in bar plots (left y-axis), and accuracy is indicated by diamond markers (right y-axis).

tation, and the limited restoration capability under aggressive quantization causes significant accuracy loss. DeepAdaIn-Net only supports splitting before the fourth branch and lacks dynamic adjustment, making it unable to adapt to bandwidth variation. Additionally, its compression and compensation also cause great losses on this dataset.

Among the collaborative methods, SP-Reconfig and DeepAdaIn-Net do not achieve optimal performance in terms of either latency or accuracy. In contrast, DMSF demonstrates the best latency-accuracy trade-off on both platforms. As shown in Fig. 4(a), at 10Mbps on the TX2 platform, DMSF achieves a latency of 65.6ms, which is significantly lower than that of the Cloud-Only method (152.6ms), representing a 57.0% reduction with only a 2.2% accuracy drop from the original 35.4%. Compared to SP-Reconfig and DeepAdaIn-Net, the latency is reduced by 32.0% and 4.2%, respectively. In terms of accuracy, DMSF achieves 33.2%, outperforming SP-Reconfig and DeepAdaIn-Net by 12.2% and 8.1%, respectively. As shown in Fig. 4(b), at 30Mbps on the Nano platform, DMSF achieves a latency of 72.9ms with accuracy 32.3%, reducing latency by 41.5% compared to the Edge-Only method (124.7ms) with only a 3.1% accuracy drop from the original 35.4%. It also outperforms SP-Reconfig and DeepAdaIn-Net, achieving 14.7% and 4.3% lower latency respectively, along with 8.4% and 5.2% higher accuracy.

V. CONCLUSION

In this paper, we proposed DMSF, a dynamic model splitting framework for edge-cloud collaborative inference. By embedding switchable compress-recover modules at split points, DMSF enables fast and low-overhead adaptation based on real-time bandwidth and edge-side computational capabilities. To mitigate information loss caused by structural simplification and feature compression, we designed hierarchical feature compensation modules to maintain perception accuracy. Experiments on multi-branch object detection model demonstrate that DMSF reduces memory overhead by 75% and achieves millisecond-level switching latency. Under 10 Mbps bandwidth on the TX2 platform, compared to the existing method, DMSF reduces latency by 32.0% and improves

perception accuracy by 12.2%. In our future research, we will explore extending DMSF to a wider range of single-branch and multi-branch models.

REFERENCES

- [1] J.-H. Kim, N. Kim, and C. S. Won, "High-speed drone detection based on yolo-v8," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 1–2, 2023.
- [2] M. Łysakowski, K. Żywanowski, A. Banaszczyk, M. R. Nowicki, P. Skrzypczyński, and S. K. Tadeja, "Real-time onboard object detection for augmented reality: Enhancing head-mounted display with yolov8," in *IEEE International Conference on Edge Computing and Communications (EDGE)*, pp. 364–371, 2023.
- [3] B. Tian, L. Wang, L. Xu, W. Pan, H. Wu, L. Li, and Z. Han, "Uav-assisted wireless cooperative communication and coded caching: A multiagent two-timescale drl approach," *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 4389–4404, 2024.
- [4] L. Wang, X. Wu, Y. Zhang, X. Zhang, L. Xu, Z. Wu, and A. Fei, "Deepadain-net: Deep adaptive device-edge collaborative inference for augmented reality," *IEEE Journal of Selected Topics in Signal Processing*, vol. 17, no. 5, pp. 1052–1063, 2023.
- [5] J. Shao and J. Zhang, "Communication-computation trade-off in resource-constrained edge inference," *IEEE Communications Magazine*, vol. 58, no. 12, pp. 20–26, 2020.
- [6] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," *ACM SIGARCH Computer Architecture News*, vol. 45, no. 1, pp. 615–629, 2017.
- [7] E. Li, L. Zeng, Z. Zhou, and X. Chen, "Edge ai: On-demand accelerating deep neural network inference via edge computing," *IEEE Transactions on Wireless Communications*, vol. 19, no. 1, pp. 447–457, 2020.
- [8] C. Hu, W. Bao, D. Wang, and F. Liu, "Dynamic adaptive dnn surgery for inference acceleration on the edge," in *IEEE Conference on Computer Communications (INFOCOM)*, pp. 1423–1431, 2019.
- [9] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2818–2826, 2016.
- [10] J. C. Lee, Y. Kim, S. Moon, and J. H. Ko, "A reconfigurable neural architecture for edge-cloud collaborative real-time object detection," *IEEE Internet of Things Journal*, vol. 9, no. 23, pp. 23390–23404, 2022.
- [11] X. Huang and S. Belongie, "Arbitrary style transfer in real-time with adaptive instance normalization," in *IEEE International Conference on Computer Vision (ICCV)*, pp. 1510–1519, 2017.
- [12] G. Jocher, "ultralytics/yolov5." <https://github.com/ultralytics/yolov5>, Oct. 2020.
- [13] D. Du, P. Zhu, L. Wen, X. Bian, H. Lin, Q. Hu, T. Peng, J. Zheng, and Wang, "Visdrone-det2019: The vision meets drone object detection in image challenge results," in *2019 IEEE/CVF International Conference on Computer Vision Workshop (ICCVW)*, pp. 213–226, 2019.